T3000 API & Protocol

Connecting Devices · built Nov 13, 2026

Connecting Devices

Learn how to discover and connect T3000 BACnet devices to your network.

Overview

The T3000 Web application provides multiple methods to discover and connect to BACnet devices on your network. This guide covers the different connection methods and best practices.

Discovery Methods

Auto Discovery

The application automatically scans your network for BACnet devices:

1. Navigate to the **Discover** page from the main menu
2. Click **Scan Network** to start the discovery process
3. Wait for the scan to complete (typically 10-30 seconds)
4. All discovered devices will appear in the device list

Supported Protocols:

- BACnet/IP (UDP port 47808)
- BACnet MSTP (serial communication)
- Modbus TCP/IP
- Modbus RTU (serial)

Manual Connection

To add a device manually:

1. Go to **Devices** > **Add Device**
2. Enter the device information:
    - **IP Address**: Device network address (e.g., 192.168.1.100)
    - **Port**: BACnet port (default: 47808)
    - **Device ID**: BACnet device instance number

- **Protocol**: Select connection protocol

3. Click **Connect**

Connection Status

Device connection states:

- 🟢 **Online**: Device is connected and responding
- 🟡 **Connecting**: Attempting to establish connection
- 🔴 **Offline**: Device is not responding
- ⚠️ **Error**: Connection error occurred

Network Configuration

IP Address Setup

Ensure your device and computer are on the same network:

```text
Device IP:     192.168.1.100
Subnet Mask:   255.255.255.0
Gateway:       192.168.1.1
```

Firewall Settings

Allow the following ports through your firewall:

- **UDP 47808**: BACnet/IP communication
- **TCP 502**: Modbus TCP
- **TCP 9103**: T3000 API server
- **TCP 3003**: T3000 Web UI

Troubleshooting

Device Not Found

1. Verify the device is powered on
2. Check network connectivity (ping the device IP)
3. Ensure firewall allows BACnet traffic
4. Verify correct subnet configuration

Connection Timeout

- Increase timeout settings in **Settings** > **Communication**
- Check for network congestion
- Verify device is not overloaded with requests

Authentication Errors

- Verify device password (if authentication enabled)
- Check user permissions
- Ensure device firmware is up to date

Best Practices

1. **Use Static IP Addresses**: Assign fixed IPs to devices for consistent connectivity
2. **Document Device IDs**: Maintain a list of device IDs and locations
3. **Regular Health Checks**: Monitor connection status regularly
4. **Backup Configuration**: Export device settings periodically
5. **Network Segmentation**: Use VLANs to organize devices by zone/function

Next Steps

- Device Configuration - Configure device settings
- Device Monitoring - Monitor device status and data

Connecting Devices

Programmatic Device Discovery

BACnet Discovery via API

TYPESCRIPT

```typescript
import { BACnetClient } from '@temco/t3000-sdk';

const client = new BACnetClient({
  port: 47808,
  broadcastAddress: '192.168.1.255',
  timeout: 30000
});

// Broadcast WhoIs request
const devices = await client.discover({
  lowLimit: 0,
  highLimit: 4194303,  // Max device instance
  networkNumber: null   // All networks
});

devices.forEach(device => {
  console.log(`Found device: ${device.deviceId} at ${device.address}`);
});
```

Modbus Discovery

TYPESCRIPT

```typescript
import { ModbusClient } from '@temco/t3000-sdk';

const client = new ModbusClient({
  host: '192.168.1.100',
  port: 502,
  timeout: 5000
});

// Test connection
const connected = await client.connect();
if (connected) {
  // Read device info
  const deviceId = await client.readHoldingRegisters(0, 1);
  console.log(`Modbus device ID: ${deviceId[0]}`);
}
```

REST API Discovery

SHELL

```
## Trigger network scan
curl -X POST http://localhost:9103/api/discovery/scan \
  -H "Content-Type: application/json" \
  -d '{
    "protocol": "bacnet",
    "subnet": "192.168.1.0/24",
    "timeout": 30000
  }'

## Get discovered devices
curl http://localhost:9103/api/discovery/devices

## Response
[
  {
    "deviceId": 389001,
    "address": "192.168.1.100:47808",
    "protocol": "bacnet",
    "modelName": "T3-BB",
    "firmwareVersion": "8.5.0"
  }
]
```

Advanced Connection Configuration

BACnet Connection Parameters

TYPESCRIPT

```typescript
interface BACnetConnectionConfig {
  address: string;
  port: number;
  deviceInstance: number;
  networkNumber?: number;
  maxApdu: 1476 | 1024 | 480 | 206 | 128 | 50;
  segmentation: 'both' | 'transmit' | 'receive' | 'none';
  apduTimeout: number;
  numberOfApduRetries: number;
  maxSegments?: number;
}

const config: BACnetConnectionConfig = {
  address: '192.168.1.100',
  port: 47808,
  deviceInstance: 389001,
  networkNumber: 0,
  maxApdu: 1476,
  segmentation: 'both',
  apduTimeout: 3000,
  numberOfApduRetries: 3,
  maxSegments: 64
};

await client.connect(config);
```

Connection Pooling

TYPESCRIPT

```typescript
class DeviceConnectionPool {
  private connections = new Map<number, BACnetClient>();
  private maxConnections = 50;

  async getConnection(deviceId: number): Promise<BACnetClient> {
    if (this.connections.has(deviceId)) {
      return this.connections.get(deviceId)!;
    }

    if (this.connections.size >= this.maxConnections) {
      // Evict least recently used
      const lru = Array.from(this.connections.keys())[0];
      this.connections.delete(lru);
    }

    const client = await this.createConnection(deviceId);
    this.connections.set(deviceId, client);
    return client;
  }

  private async createConnection(deviceId: number): Promise<BACnetClient> {
    const device = await this.lookupDevice(deviceId);
    return new BACnetClient({
      address: device.address,
      port: device.port,
      deviceInstance: deviceId
    });
  }
}
```

Connection Monitoring

Health Checks

TYPESCRIPT

```typescript
class DeviceHealthMonitor {
  async checkDevice(deviceId: number): Promise<HealthStatus> {
    const start = Date.now();

    try {
      // Attempt read of system status object
      const status = await client.readProperty({
        objectType: 'device',
        objectInstance: deviceId,
        property: 'system-status'
      });

      const latency = Date.now() - start;

      return {
        online: true,
        latency,
        systemStatus: status.value,
        lastCheck: new Date()
      };
    } catch (error) {
      return {
        online: false,
        error: error.message,
        lastCheck: new Date()
      };
    }
  }

  async monitorDevices(deviceIds: number[], interval: number) {
    setInterval(async () => {
      for (const deviceId of deviceIds) {
        const health = await this.checkDevice(deviceId);
        await this.updateDatabase(deviceId, health);

        if (!health.online) {
```

```
          await this.sendAlert(deviceId, health);
        }
      }
    }, interval);
  }
}
```

Connection Retry Logic

TYPESCRIPT

```typescript
async function connectWithRetry(
  config: ConnectionConfig,
  maxRetries: number = 3,
  backoff: number = 1000
): Promise<BACnetClient> {
  let lastError: Error;

  for (let attempt = 0; attempt < maxRetries; attempt++) {
    try {
      const client = new BACnetClient(config);
      await client.connect();
      return client;
    } catch (error) {
      lastError = error;
      const delay = backoff * Math.pow(2, attempt);
      console.log(`Retry ${attempt + 1}/${maxRetries} after ${delay}ms`);
      await sleep(delay);
    }
  }

  throw new Error(`Failed after ${maxRetries} attempts: ${lastError}`);
}
```

Network Discovery Protocols

BACnet WhoIs/IAm

TYPESCRIPT

```typescript
class BACnetDiscovery {
  async sendWhoIs(lowLimit?: number, highLimit?: number) {
    const packet = encodeBACnetPacket({
      pduType: 'UnconfirmedRequest',
      service: 'who-is',
      lowLimit: lowLimit ?? 0,
      highLimit: highLimit ?? 4194303
    });

    await this.socket.send(packet, 47808, '255.255.255.255');
  }

  onIAm(callback: (device: DeviceInfo) => void) {
    this.socket.on('message', (msg, rinfo) => {
      const packet = decodeBACnetPacket(msg);

      if (packet.service === 'i-am') {
        callback({
          deviceId: packet.deviceInstance,
          address: rinfo.address,
          port: rinfo.port,
          maxApdu: packet.maxApduLength,
          segmentation: packet.segmentationSupported,
          vendorId: packet.vendorId
        });
      }
    });
  }
}
```

Modbus Device Scanning

TYPESCRIPT

```typescript
async function scanModbusDevices(
  baseIp: string,
  startSlaveId: number,
  endSlaveId: number
): Promise<ModbusDevice[]> {
  const devices: ModbusDevice[] = [];

  for (let slaveId = startSlaveId; slaveId <= endSlaveId; slaveId++) {
    try {
      const client = new ModbusClient({
        host: baseIp,
        port: 502,
        unitId: slaveId,
        timeout: 2000
      });

      await client.connect();

      // Read device identification
      const registers = await client.readHoldingRegisters(0, 10);

      devices.push({
        slaveId,
        address: baseIp,
        registers: registers
      });

      await client.disconnect();
    } catch (error) {
      // Device not responding at this slave ID
      continue;
    }
  }

  return devices;
}
```

Database Schema for Devices

SQL

```sql
CREATE TABLE devices (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  serial_number INTEGER UNIQUE NOT NULL,
  ip_address TEXT,
  port INTEGER,
  protocol TEXT CHECK(protocol IN ('bacnet', 'modbus')),
  device_instance INTEGER,
  model_name TEXT,
  firmware_version TEXT,
  status TEXT CHECK(status IN ('online', 'offline', 'error')),
  last_seen INTEGER,
  created_at INTEGER DEFAULT (strftime('%s', 'now')),
  updated_at INTEGER DEFAULT (strftime('%s', 'now'))
);

CREATE INDEX idx_devices_serial ON devices(serial_number);
CREATE INDEX idx_devices_status ON devices(status);
CREATE INDEX idx_devices_protocol ON devices(protocol);

-- Connection history
CREATE TABLE connection_logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  device_id INTEGER,
  event_type TEXT CHECK(event_type IN ('connect', 'disconnect', 'error')),
  message TEXT,
  timestamp INTEGER DEFAULT (strftime('%s', 'now')),
  FOREIGN KEY (device_id) REFERENCES devices(id)
);
```

WebSocket Real-Time Connection Status

TYPESCRIPT

```
// Subscribe to connection events
const ws = new WebSocket('ws://localhost:9103/ws');

ws.send(JSON.stringify({
  type: 'subscribe',
  channel: 'device-status'
}));

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);

  if (data.type === 'device-status-change') {
    console.log(`Device ${data.deviceId}: ${data.status}`);
    updateUI(data.deviceId, data.status);
  }
};
```

Next Steps

- Device Configuration API
- REST API Reference
- WebSocket API
- Modbus Protocol

Overview

The T3000 Web application provides multiple methods to discover and connect to BACnet devices on your network. This guide covers the different connection methods and best practices.

Discovery Methods

Auto Discovery

The application automatically scans your network for BACnet devices:

1. Navigate to the **Discover** page from the main menu
2. Click **Scan Network** to start the discovery process
3. Wait for the scan to complete (typically 10-30 seconds)
4. All discovered devices will appear in the device list

Supported Protocols:

- BACnet/IP (UDP port 47808)
- BACnet MSTP (serial communication)
- Modbus TCP/IP
- Modbus RTU (serial)

Manual Connection

To add a device manually:

1. Go to **Devices** > **Add Device**
2. Enter the device information:
   - **IP Address**: Device network address (e.g., 192.168.1.100)
   - **Port**: BACnet port (default: 47808)
   - **Device ID**: BACnet device instance number
   - **Protocol**: Select connection protocol
3. Click **Connect**

Connection Status

Device connection states:

- 🟢 **Online**: Device is connected and responding
- 🟡 **Connecting**: Attempting to establish connection
- 🔴 **Offline**: Device is not responding
- ⚠️ **Error**: Connection error occurred

Network Configuration

IP Address Setup

Ensure your device and computer are on the same network:

```
TEXT

Device IP:     192.168.1.100
Subnet Mask:   255.255.255.0
Gateway:       192.168.1.1
```

Firewall Settings

Allow the following ports through your firewall:

- **UDP 47808**: BACnet/IP communication
- **TCP 502**: Modbus TCP
- **TCP 9103**: T3000 API server
- **TCP 3003**: T3000 Web UI

Troubleshooting

Device Not Found

1. Verify the device is powered on
2. Check network connectivity (ping the device IP)
3. Ensure firewall allows BACnet traffic
4. Verify correct subnet configuration

Connection Timeout

- Increase timeout settings in **Settings** > **Communication**
- Check for network congestion
- Verify device is not overloaded with requests

Authentication Errors

- Verify device password (if authentication enabled)
- Check user permissions
- Ensure device firmware is up to date

Best Practices

1. **Use Static IP Addresses**: Assign fixed IPs to devices for consistent connectivity
2. **Document Device IDs**: Maintain a list of device IDs and locations
3. **Regular Health Checks**: Monitor connection status regularly
4. **Backup Configuration**: Export device settings periodically
5. **Network Segmentation**: Use VLANs to organize devices by zone/function

Next Steps

- Device Configuration - Configure device settings
- Device Monitoring - Monitor device status and data
- Troubleshooting - Resolve common issues